

Время жизни переменной — это время, в течение которого переменная связана с определенной областью памяти (*более подробную информацию см. в разд. 1.5*).

Область видимости переменной — это последовательность операторов программы, из которых можно обратиться к этой переменной (*более подробную информацию см. в разд. 1.6*).

1.4. Механизмы типизации

Типы могут определяться статически и динамически. При *статическом* определении типа связывание осуществляется при трансляции программы, а при *динамическом* — во время выполнения программы.

1.4.1. Статические и динамические типы данных

Статическое определение типа может быть выполнено путем явного или неявного объявления переменных. *Явное объявление* имеет место, если в программу включен специальный оператор объявления типа, устанавливающий тип для переменных, перечисленных в операторе. *Неявное объявление* связывает переменные с типами посредством принятых по умолчанию соглашений. При этом первое появление имени в программе расценивается как объявление переменной.

Большинство языков программирования требует явного объявления переменных. В случае неявного объявления в языке должны быть заданы правила, с помощью которых определяется тип переменных. Например, в программах на языке FORTRAN, если идентификатор не был объявлен явно, по умолчанию считается, что он имеет тип INTEGER, если он начинается с одной из букв I, J, K, L, M или N; в противном случае идентификатор является именем переменной типа REAL. В языке Perl имена, начинающиеся с символа '@', являются именами массивов, а идентификаторы, начинающиеся с символа '\$', могут именоваться только переменные числового и строкового типов. Неявные объявления ухудшают надежность программ, т. к. необъявленные по ошибке переменные будут неправильно связаны с типами, устанавливаемыми по умолчанию, что может привести к непредсказуемым ошибкам, которые сложно обнаружить.

При динамическом связывании переменная связывается с типом в момент присваивания переменной значения. При этом тип переменной будет идентичен типу присваиваемого значения. Основным преимуществом динамического связывания переменных с типом является то, что в этом случае обеспечивается высокая гибкость языка. Например, программист может написать программу сортировки элементов массива, который может содержать данные любого типа. Переменные, предназначенные для хранения элементов массива, будут связываться с соответствующим типом во время

ввода данных. Классическим примером динамического связывания переменных с типом является язык APL, в котором корректной является следующая последовательность операторов:

```
A ← 1, 100, 1000
```

```
A ← 2.5
```

Независимо от предыдущего значения типа переменная *A* после выполнения первого оператора присваивания будет представлять собой целочисленный массив, содержащий 3 элемента: 1, 100 и 1000. В результате выполнения второго оператора присваивания переменная *A* станет простой переменной, содержащей вещественное число 2,5.

В языках функционального программирования (ML, Miranda, Haskell) распространен механизм логического вывода типа. Например, в языке ML объявление функции:

```
Fun circ_length (r) = 6.28318 * r;
```

определяет функцию, аргумент и результат которой имеют вещественный тип. Вещественный тип логически выводится из типа константы, входящей в выражение.

Динамическое связывание типов имеет ряд недостатков:

- ❑ снижается возможность обнаружения транслятором ошибок по сравнению со статическим связыванием типов, т. к. при динамическом связывании во время трансляции отсутствует информация о типе переменных;
- ❑ при реализации динамического связывания вся информация о типах переменных должна сохраняться в течение всего времени работы программы, что требует значительных дополнительных ресурсов памяти, связанных с необходимостью хранить данные различных типов;
- ❑ динамическое связывание типов приводит к увеличению времени работы программы за счет программной реализации механизмов связывания.

Языки программирования с динамическим механизмом связывания типов часто реализуются в виде интерпретаторов в связи со сложностью реализации динамического изменения типов на машинном языке.

Ключевым фактором, обеспечивающим необходимый уровень надежности языка программирования, является механизм типизации, реализованный в языке.

Различают следующие механизмы типизации:

- ❑ слабая типизация;
- ❑ строгая типизация.

1.4.2. Слабая типизация

В языках программирования со слабой типизацией информация о типе данных используется только для обеспечения корректности программ на машинном уровне.

Выделяют следующие недостатки слабой типизации:

1. Операция, которая может восприниматься компьютером как корректная, может быть некорректной на абстрактном уровне программы.

Пусть имеется фрагмент программы на языке C:

```
char c;  
c = 7;
```

На машинном уровне символы представляются целыми числами от 0 до 255, поэтому приведенный оператор присваивания корректен. Если программист на абстрактном уровне программы допустил ошибку, собираясь написать `c = '7'`, то при слабой типизации такого рода ошибки не отслеживаются.

2. При работе с разными типами слабо типизированный язык предусматривает автоматическое выполнение операций преобразования типа.

Рассмотрим фрагмент программы на языке C:

```
int i;  
float x;  
...  
i = x;
```

Приведенный фрагмент программы синтаксически корректен, но требует преобразования вещественной переменной `x` из ее внутреннего представления с плавающей точкой в целое представление. Если при этом значение `x` выходит за пределы области допустимых целых чисел, реализуемых компилятором, то на этапе выполнения программы будет обнаружена ошибка.

Если переменные `i` и `k` — целого типа, а `x` — вещественного, то для определения последовательности, в которой будут выполняться операции преобразования типа при выполнении в языке C оператора присваивания

```
k = x - i;
```

необходимо знать, какой порядок преобразования типов определен в компиляторе. Возможны следующие варианты:

- `x` преобразуется к целому типу;
- `i` преобразуется к вещественному типу, а затем `x - i` — к целому.

3. Слабая типизация позволяет определять для некоторых типов некорректные операции.

Например, в языке C можно использовать для целых операндов логические операции. Если длина слова равна 16 битам, оператор $i = (k \ll 12) \mid 1$ упаковывает значение k в четырех левых разрядах i и засылает 1 в крайний правый разряд слова.

Замечание

Увеличение гибкости, обеспечиваемое слабой типизацией, приводит к уменьшению удобочитаемости программы и к необходимости дополнительного контроля во время выполнения программы.

1.4.3. Строгая типизация

Строгая типизация является крайне полезным механизмом типизации, который обеспечивает полный контроль типов (статический и динамический). Строгая типизация существенно повышает надежность и ясность программы, т. к. всем операциям обеспечивается корректность на абстрактном уровне языка.

В строго типизированном языке:

- ☐ каждый объект данных обладает уникальным типом;
- ☐ каждый тип определяет множество значений и множество операций;
- ☐ в каждой операции присваивания тип присваиваемого значения и тип переменной, которой присваивается значение, должны быть эквивалентны;
- ☐ каждая примененная к объекту данных операция должна принадлежать множеству операций, допустимых для объектов данного типа;
- ☐ преобразование типа должно задаваться явно, например $i = (\text{int})\ x$.

1.4.4. Производные типы

Если в языке имеются только predefined типы, то надежность такого языка невелика вследствие малого числа и широкого диапазона значений predefined типов. Существенный результат при разработке механизма строгой типизации можно получить путем введения в язык типов данных, определенных пользователем.

Тип данных можно рассматривать как *факторизацию* определенных свойств, являющихся общими для некоторого класса объектов. Если множество типов данных ограничено predefined типами, то все объекты, которые представлены, например, вещественным типом, должны принадлежать

одному и тому же классу, что может не соответствовать смыслу и использованию таких объектов в программе.

Программа на языке Pascal, читающая текущее значение веса товара и вычисляющая суммарный вес десяти товаров, представлена в листинге 1.1.

Листинг 1.1

```
program sum (input, output);  
  var  
    temp_weight, sum_weight: integer;  
    i: integer;  
begin  
  sum_weight := 0;  
  for i := 1 to 10 do  
    begin  
      read (temp_weight);  
      sum_weight := sum_weight + temp_weight  
    end;  
    writeln (sum_weight)  
  end.
```

В этой программе переменные `temp_weight`, `sum_weight` и `i` являются *целыми*, хотя и принадлежат к двум логически разным классам объектов: вес и индекс. Если по ошибке внутри цикла написать оператор:

```
sum_weight := sum_weight + i;
```

то компилятор не сможет ее обнаружить, т. к. переменные `sum_weight` и `i` одного типа. Для того чтобы компилятор выявлял семантические ошибки такого типа, многие языки программирования разрешают определять в программе собственные (*производные*) типы на основе базовых (*порождающих*) типов. Определим новые типы `weight` (вес) и `index` (индекс), используя базовый тип `integer`. Теперь описанные в программе переменные типа `weight` будут логически отличаться от переменных типа `index`.

С использованием производных типов `weight` и `index` приведенная ранее программа может быть переписана так, как приведено в листинге 1.2.

Введение в язык производных типов дает возможность компилятору обнаруживать ошибки вида `sum_weight := sum_weight + i`. Обнаруживается ли такая ошибка на самом деле, зависит от реализованного в компиляторе метода определения эквивалентности типов. При реализации производных типов необходимо также решить вопрос: какие атрибуты производный тип должен наследовать от порождающего типа.

Листинг 1.2

```
program sum1 (input, output);
  type
    weight = integer;  {вес}
    index = integer;   {индекс}
  var
    temp_weight, sum_weight: weight;
    i: index;
begin
  sum_weight := 0;
  for i := 1 to 10 do
  begin
    read (temp_weight);
    sum_weight := sum_weight + temp_weight
  end;
  writeln (sum_weight)
end.
```

1.4.5. Эквивалентность типов

Возможны два метода определения эквивалентности типов:

- ☐ с использованием структурной эквивалентности;
- ☐ с использованием именной эквивалентности.

При использовании *структурной эквивалентности* два объекта принадлежат эквивалентным типам, если у них одинаковая структура. Фактически при структурной эквивалентности производные типы являются синонимами для имени порождающего типа, и компилятор должен разрешать присваивания типа `sum_weight := i`, т. к. структурно `sum_weight`, и `i` представляются как целые. Для производных типов структурная эквивалентность обеспечивает бо́льшую наглядность и ясность программы, не увеличивая ее надежности.

При использовании *именной эквивалентности* два объекта принадлежат эквивалентным типам только в том случае, если они описаны с помощью одного и того же имени типа. При использовании именной эквивалентности компилятор не будет допускать операторы вида `sum_weight := i`. Именная эквивалентность является гораздо более ограничивающей формой строгой типизации, чем структурная.

Преимущества именной эквивалентности заключаются в следующем:

- ☐ именная эквивалентность обеспечивает бо́льшую надежность компилятора;

- компилятор упрощается за счет того, что для именной эквивалентности не нужно разрабатывать алгоритмы сравнения шаблонов, необходимые при реализации структурной эквивалентности для сложных структурных типов данных.

Можно указать на следующие недостатки именной эквивалентности:

- необходимость явного определения функций преобразования типа;
- ограничение возможности использования описаний *анонимного* типа (подробно данный вопрос будет рассмотрен в *разд. 1.4.8*).

1.4.6. Наследование атрибутов

Множество атрибутов типа включает в себя:

- множество значений;
- литеральные обозначения констант;
- множество определенных для типа операций.

При использовании структурной эквивалентности производный тип может унаследовать все атрибуты порождающего типа. В случае именной эквивалентности производный тип наследует от порождающего типа множество значений и литеральные обозначения констант, но, как правило, может наследовать не все его операции.

Рассмотрим программу вычисления площади прямоугольника (листинг 1.3).

Листинг 1.3

```
program square_figure (input, output);  
  type  
    length = real;    {длина}  
    square = real;    {площадь}  
  var  
    a, b: length;  
    s: square;  
begin  
  read (a, b);  
  s := a * b;  
  writeln (s)  
end.
```

Если операция умножения наследуется от вещественного типа, то разумно предположить, что и тип результата операции $a * b$ совпадает с типом

операндов, т. е. `length`. Следовательно, оператор присваивания `s := a * b` недопустим!

Можно использовать функцию преобразования типа `s := square(a * b)`, но это не логично, т. к. назначение производных типов состоит в запрещении некорректных операций, а не в ограничении корректных. С другой стороны, оператор `a := a * b` синтаксически корректен, но является бессмысленным с точки зрения размерности.

Замечание

Для обеспечения высокой надежности за счет использования производных типов язык должен обладать механизмом *переопределения существующих операций и механизмом определения новых операций*.

В современных языках программирования операции, как правило, *перегружены*, т. е. операции для различных типов операндов имеют одинаковое обозначение. Например, выполнение операции сложения целых чисел отличается от выполнения операции сложения вещественных чисел, хотя имеют одно и то же обозначение — знак '+'. Тип операции в каждом конкретном случае определяется компилятором путем исследования типов операндов и предполагаемого результата.

Обычно операция присваивания и операции отношения "равно" и "не равно" *корректны для любого типа*. Для других же операций необходимо предусмотреть механизм, с помощью которого множество операций, наследуемое от порождающего типа, должно ограничиваться, а затем расширяться за счет определенных пользователем операций.

1.4.7. Ограничения

Для повышения надежности языка обычно ограничивают не только множество операций, наследуемое от базового типа, но и наследуемое множество значений.

Ограничение диапазона можно включить непосредственно в определение производного типа, например:

```
type index = 1..100;
```

В этом случае тип `index` определяет множество целых значений в диапазоне от 1 до 100.

Ограничение множества значений, наследуемого производным типом, до множества необходимых значений повышает ясность и надежность программы и оптимизирует результирующую программу за счет исключения проверок принадлежности значений переменных заданному диапазону.